

Assignment 1 - Git & GitHub Fundamentals

Report

Full Name: Mtouri Mohammed Taha

Student ID: 102913

Course Code & Section: CSC 3324, Section 02

GitHub Username: tahamt4

1. Repository Setup

I created the folder git-session1-tahamt4, copied the STARTER files into it and ran git init. The default branch was main. In .gitignore I ignore local.env (machine-specific settings that must not be committed) and all .log files with *.log, and I added the exception !important.log so that file stays visible to Git. I checked the rules with git status and git check-ignore -v: local.env, app.log and debug.log are ignored, while important.log is listed as untracked.

```
Mtouri Taha@Taha-Elitebook UCRT64 ~/OneDrive/Documents/Software_Engineering/git-session1-tahamt4 (main)
$ git status
git check-ignore -v local.env debug.log app.log
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        app.js
        assets/
        important.log
        index.html
        notes.txt
        style.css

nothing added to commit but untracked files present (use "git add" to track)
.gitignore:2:local.env    local.env
.gitignore:5:*.log       debug.log
.gitignore:5:*.log       app.log

Mtouri Taha@Taha-Elitebook UCRT64 ~/OneDrive/Documents/Software_Engineering/git-session1-tahamt4 (main)
```

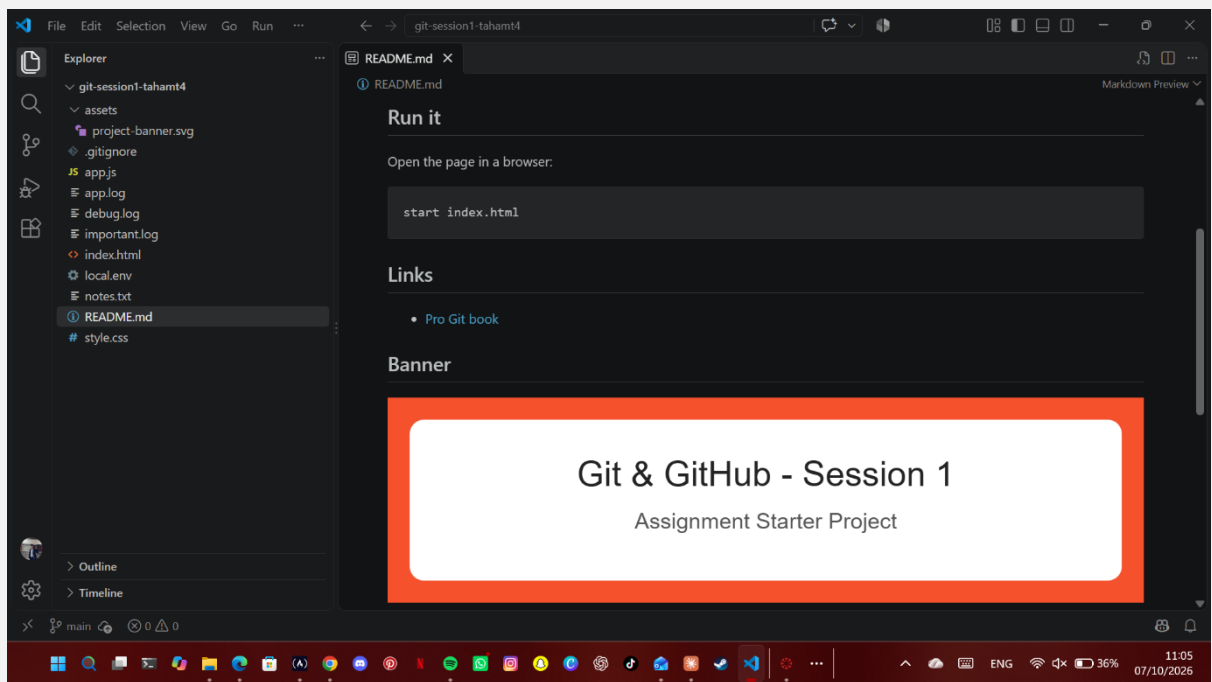
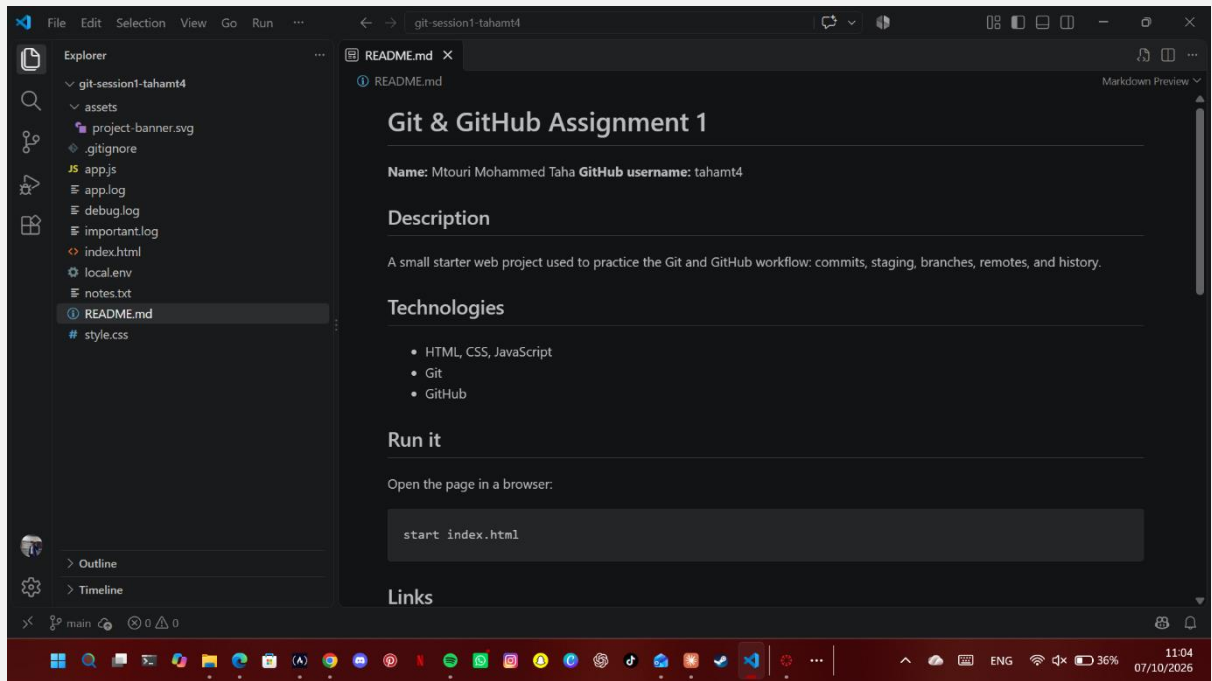
Screenshot 1: final ignore state (git status and git check-ignore -v)

2. README & Commit History

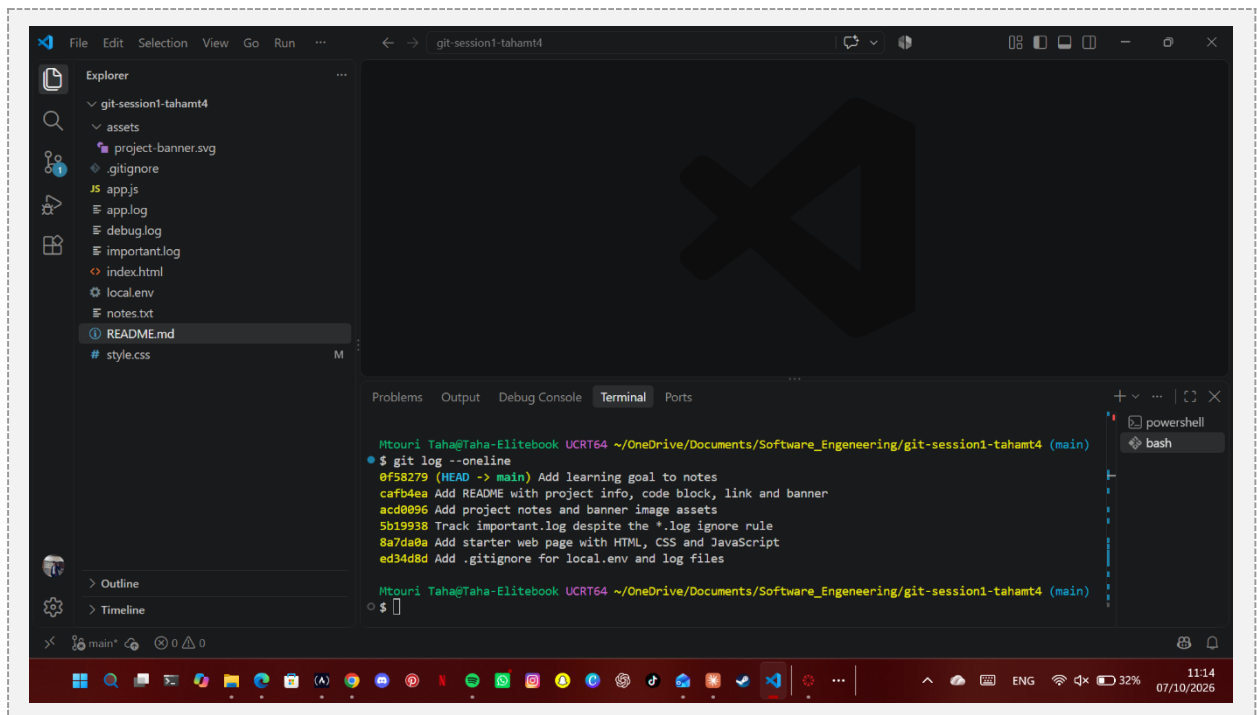
The README.md contains my name, my GitHub username, a short description, the technologies, one code block, one link (Pro Git book) and the provided banner image from assets/. I built the history in small logical commits, each one adding a single group of related files:

- ed34d8d Add .gitignore for local.env and log files
- 8a7da0a Add starter web page with HTML, CSS and JavaScript
- 5b19938 Track important.log despite the *.log ignore rule
- acd0096 Add project notes and banner image assets
- cafb4ea Add README with project info, code block, link and banner

Further commits (selective staging, About feature, merge) are visible in the history below.



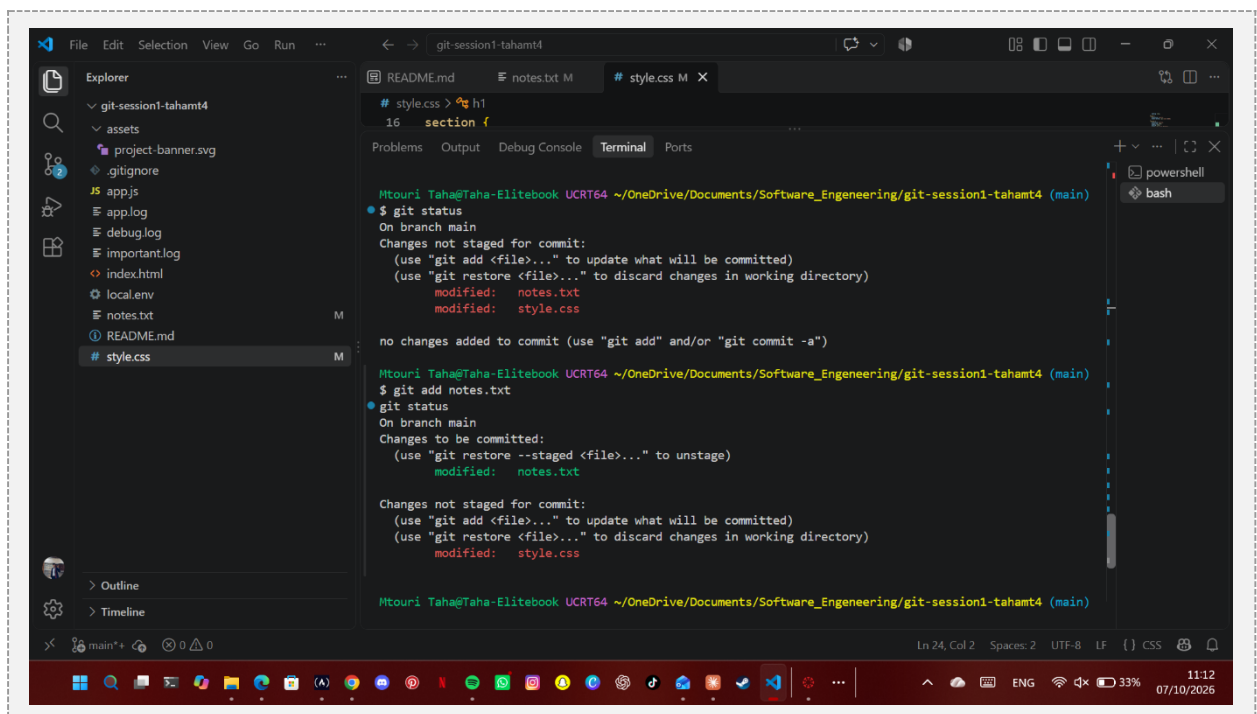
Screenshot 2: rendered README



Screenshot 3: compact commit history

3. Selective Staging

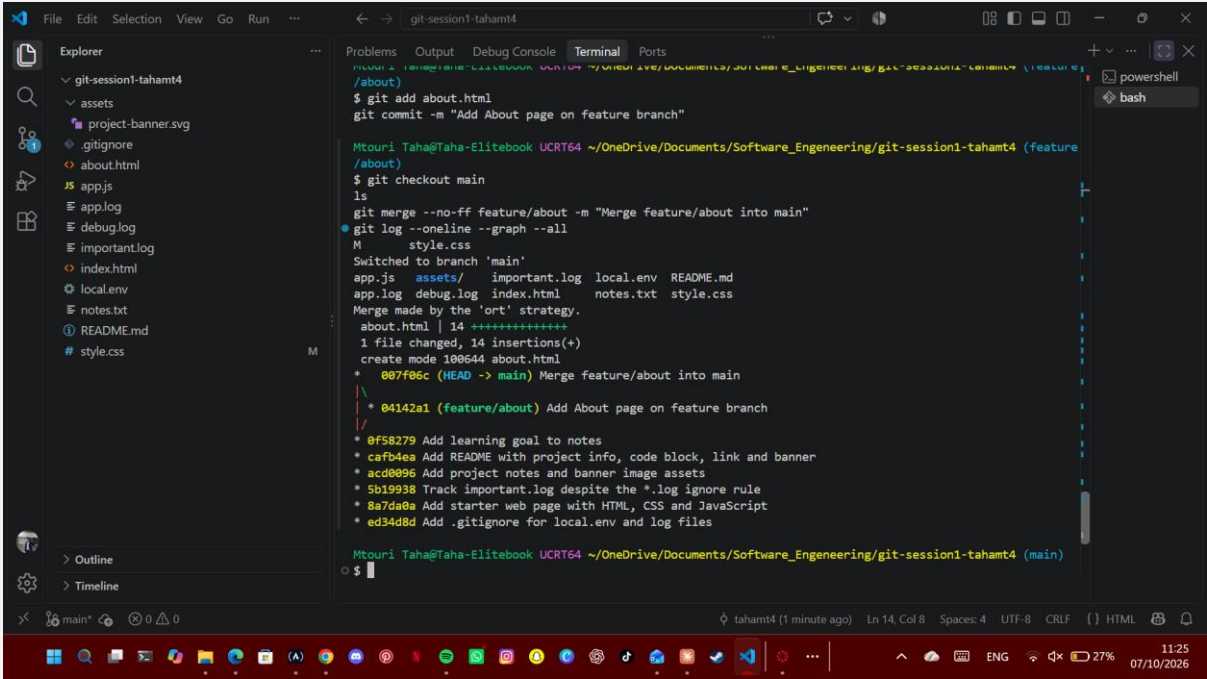
I edited two tracked files by hand in VS Code: I added a line to notes.txt and an h1 rule to style.css. git status listed both under "Changes not staged for commit". I ran git add notes.txt so only that file was staged, and git status then showed notes.txt under "Changes to be committed" while style.css stayed unstaged. I committed with git commit -m "Add learning goal to notes" (0f58279). After the commit, style.css was still modified.



Screenshot 4: one file staged, the other left modified

4. Branch & Merge

I created the feature branch with `git checkout -b feature/about`, added `about.html` and committed it there (04142a1). I returned to main with `git checkout main`, where `about.html` was not present, and merged with `git merge --no-ff feature/about`. The `--no-ff` option creates a merge commit (007f06c), so the branch stays visible in the history graph.



The screenshot shows a VS Code terminal window with the following content:

```
git-session1-tahamt4
/feature/about
$ git add about.html
git commit -m "Add About page on feature branch"

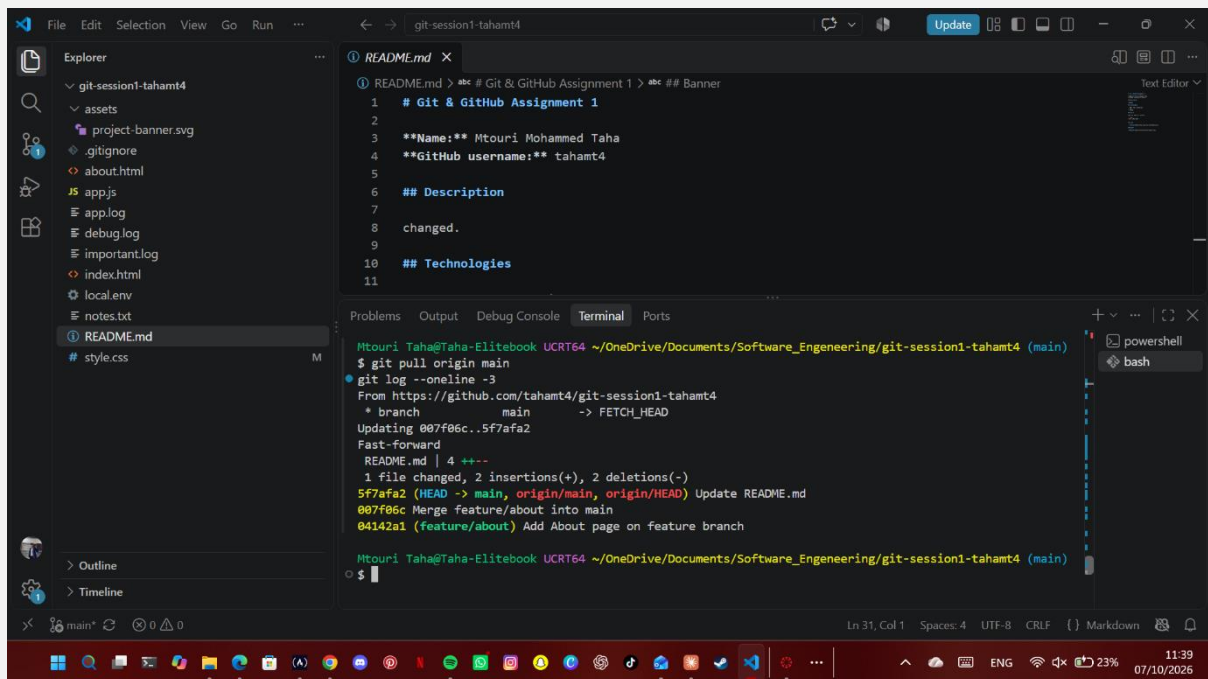
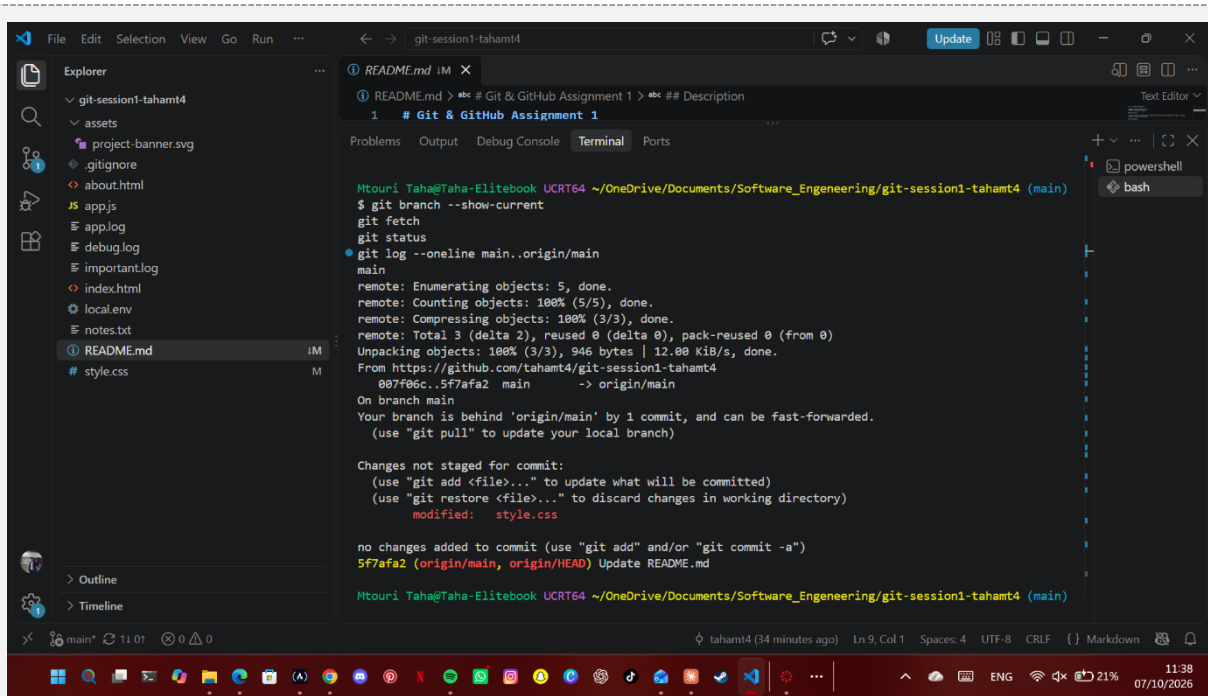
Mtourri Taha@Taha-Elitebook UCR764 ~/OneDrive/Documents/Software_Engineering/git-session1-tahamt4 (feature/about)
$ git checkout main
ls
git merge --no-ff feature/about -m "Merge feature/about into main"
git log --oneline --graph --all
M style.css
Switched to branch 'main'
app.js assets/ important.log local.env README.md
app.log debug.log index.html notes.txt style.css
Merge made by the 'ort' strategy.
about.html | 14 ++++++
1 file changed, 14 insertions(+)
create mode 100644 about.html
* 007f06c (HEAD -> main) Merge feature/about into main
|
| * 04142a1 (feature/about) Add About page on feature branch
|
| * 0f58279 Add learning goal to notes
| * cafb4ea Add README with project info, code block, link and banner
| * acd0096 Add project notes and banner image assets
| * 5b19938 Track important.log despite the *.log ignore rule
| * 8a7da0a Add starter web page with HTML, CSS and JavaScript
| * ed34d8d Add .gitignore for local.env and log files
|
Mtourri Taha@Taha-Elitebook UCR764 ~/OneDrive/Documents/Software_Engineering/git-session1-tahamt4 (main)
$
```

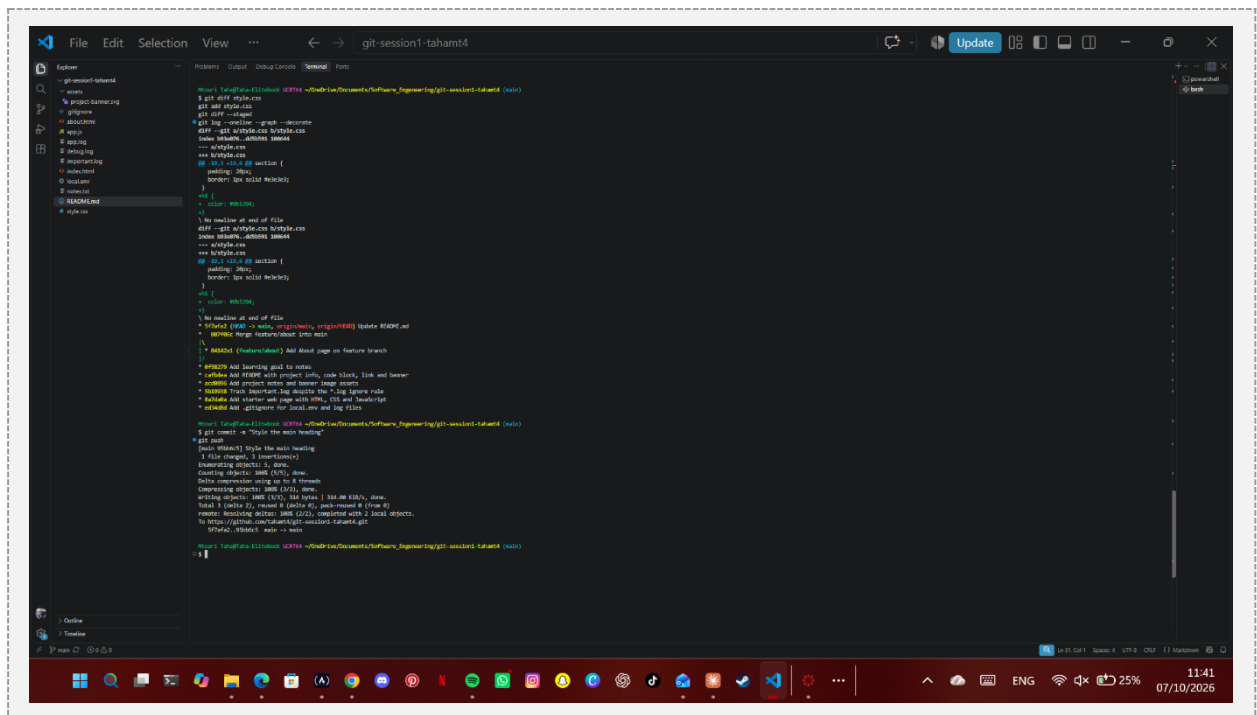
Screenshot 5: branch and merge history

5. Remote Changes

A commit saves a snapshot of the staged changes in my local repository only. A push uploads my local commits to the remote repository on GitHub, so commits exist locally before they exist online. `git fetch` downloads new commits and updates the remote-tracking branch (origin/main) without touching my working files. `git pull` does a fetch and then merges the new commits into my current branch, so it changes my local files.

I created an empty GitHub repository, connected it with `git remote add origin` and published main with `git push -u origin main`. I then edited `README.md` directly on GitHub (commit 5f7afa2) where I changed the description from a sentence to "changed". After `git fetch`, `git status` reported that local main was behind origin/main by 1 commit, and my local files were unchanged. `git pull origin main` brought the change in as a fast-forward. Finally I showed the unstaged diff (`git diff style.css`), the staged diff (`git diff --staged`, after `git add style.css`) and the history (`git log --oneline --graph --decorate`).





Screenshot 6: after fetch, local main is behind origin/main

6. Fair Use of AI

I uploaded the assignment to Claude and asked it to guide me step by step, and I asked questions like the -b option of git init does, what each starter file is for, and how to stage only one file. I followed the steps using the commands we learned in the class presentation, and I changed them where needed: I used plain git init, I edited files by hand in VS Code instead of using echo and printf, and I used git checkout instead of git switch, as in class. Claude also gave me a template for this report so I could add my screenshots. I checked every step by running the commands myself and reading the output of git status, git log and git check-ignore, and I understood all the commands.

7. Sources & Reflection

- Session 1 slides - Git Basics & Your First GitHub Repository (Group B)
- Assignment 1 instructions and report template (Group B)

The hardest concept for me was the staging area: a file can be modified, staged and committed as three different states, and git add decides what goes into the next commit. If Git behaves unexpectedly, I first run git status, then git log --oneline --graph --all, to see which branch I am on and what is staged before I try anything else.